



Università degli Studi di Padova

Dipartimento di Matematica "Tullio Levi-Civita"

Corso di Laurea in Informatica

Evoluzione e ottimizzazione di un firmware embedded per un sistema di telemetria basato su ESP32

EggLog Motion Core – telemetria per il motorsport motociclistico

Zakaria Laoud

Relatore: Marco Zanella

Anno Accademico 2025/2026

EggLog Motion Core: l'idea del progetto



Obiettivi

- ▶ Ottimizzazione del firmware (RAM & CPU)
- ▶ Evoluzione con nuove funzionalità
- ▶ Analisi di fattibilità del **EggLog CAN Sniffer**

- ▶ **Software house** padovana
- ▶ Nata grazie all'esperienza **USA**
- ▶ Team **giovane**

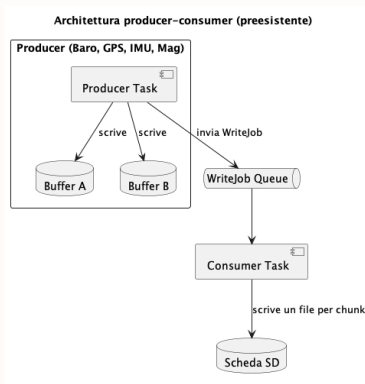
Situazione di partenza: firmware

Sistema esistente

- ▶ Architettura **producer-consumer** con **ping-pong buffer**
- ▶ Salvataggio di un file **CSV** per buffer riempito

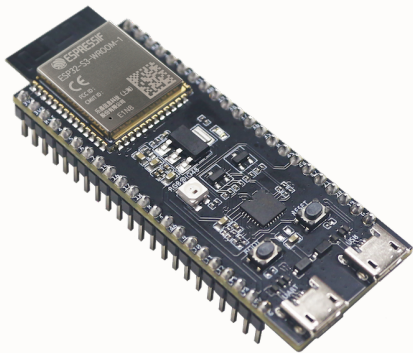
Vincoli di progetto

- ▶ Firmware in **C++** su microcontrollori **ESP32**
- ▶ **Sniffer CAN** separato in modalità **listen-only** sul bus della moto



Producer-consumer con ping-pong buffering

Tecnologie utilizzate



ESP32-S3-DevKitC-1

- ▶ **ESP32-S3/-WROOM-32** – microcontrollore principale/sniffer can
- ▶ **Arduino Core** – framework per lo sviluppo del firmware su ESP32
- ▶ **FreeRTOS** – sistema operativo per la gestione dei task concorrenti
- ▶ **PlatformIO** – ambiente di build e gestione delle dipendenze

Situazione di partenza: applicazione

Sistema esistente

- ▶ Visualizzazione dei dati in tempo reale
- ▶ Nessun **modello 3D** del veicolo
- ▶ Configurazione dei **parametri** limitata

Vincoli di progetto

- ▶ Piattaforma **React Native / TypeScript**
- ▶ Comunicazione col device solo via **BLE**

Problemi iniziali da risolvere

Storage frammentato

Tanti file CSV rallentano la **ricerca dei cluster liberi**

Formato inefficiente

CSV: elevato spazio occupato e **recovery** complesso

Sessioni non gestite

Nessuna organizzazione in **sessioni**

Stato non visibile

Nessuno stato centrale **interno** né **LED** esterno

Dati IMU e Barometro grezzi

Tutti i dati vengono inviati, **anche se privi di significato**

App incompleta

Nessun **modello 3D** del veicolo

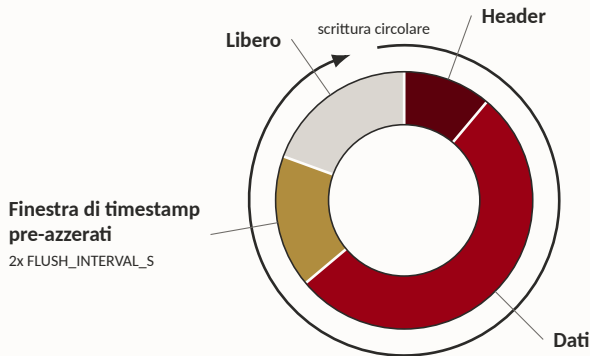
Dati ECU assenti

Nessun accesso ai dati del **bus CAN**

Superfile: il nuovo sistema di persistenza

Risolve: storage frammentato, formato inefficiente e l'assenza di recovery

- Sostituisce **tanti file CSV** con **un unico file**
- **Buffer circolare** preallocato: zero frammentazione
- Formato **binario compatto**: più campioni per byte
- **Finestra pre-azzerata**: recovery automatico dopo crash



Stato del dispositivo, LED e gestione sessione

Risolve: stato del firmware non visibile

Segnalazioni visive del dispositivo

 SD mancante	 Errore	 Pairing BLE
 Pairing Wi-Fi	 Recupero dati	 Calibrazione
 Download	 Cancellazione	 Sospensione
 Logging	 Stand-by	

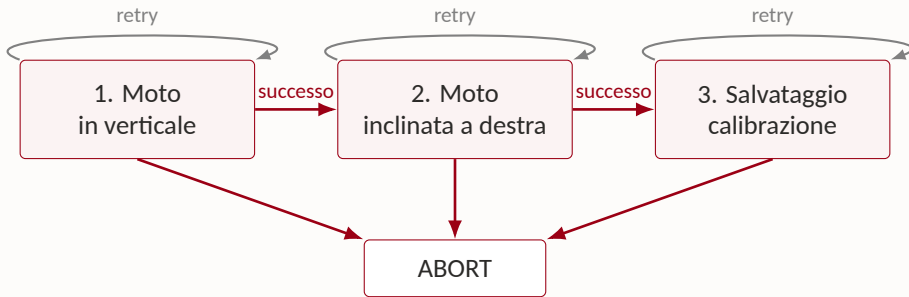
Priorità deterministica in caso di stati concorrenti

- ▶ Stato del dispositivo **centralizzato**
- ▶ Stati firmware: **STAND_BY, LOGGING, CALIBRATING, PAIRING...**
- ▶ **Colore e pattern di illuminazione** specifici per ogni stato

Calibrazione dei sensori

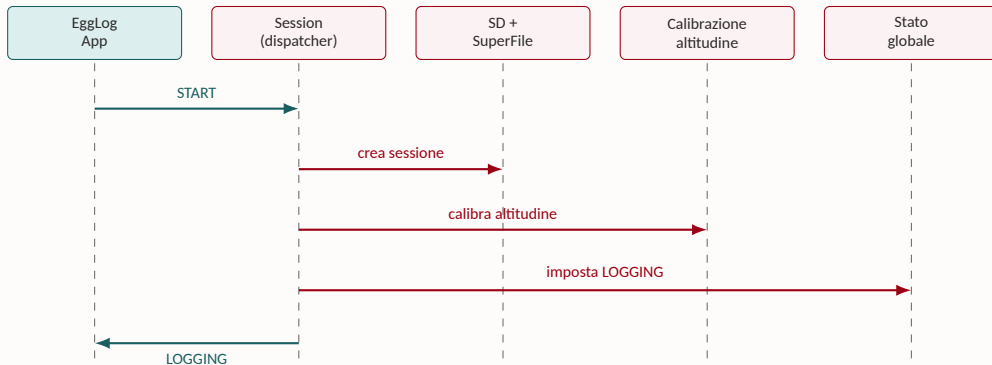
Risolve: dati IMU senza significato

- ▶ **Barometro:** calibrazione **altitudine** ad inizio sessione
- ▶ Precisione **2 m** vs **25 m** GPS
- ▶ **IMU:** calibrazione guidata in **3 passaggi**
- ▶ Macchina a stati con **retry isolato**



Gestione delle sessioni

Risolve: sessioni non gestite



- Ogni comunicazione avviene mediante **BLE** tra firmware e applicazione

EggLog CAN Sniffer

Risolve: dati ECU mancanti

Nodo **separato**, non collegato al microcontrollore centrale



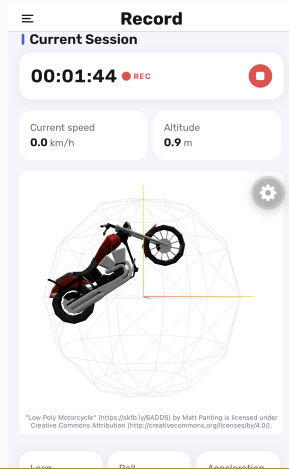
Problemi risolti:

- ▶ **Transienti elettrici** → TVS in serie
- ▶ **Interferenza bus** → controller **listen-only**
- ▶ **Massa flottante** → collegamento a Y

Applicazione mobile: nuove funzionalità

Risolve: app incompleta (modello 3D e avviso stato SD)

- ▶ **Modello 3D** del veicolo in tempo reale
- ▶ Avviso di **microSD** mancante
- ▶ Visualizzazione dello **spazio occupato** dei superfile

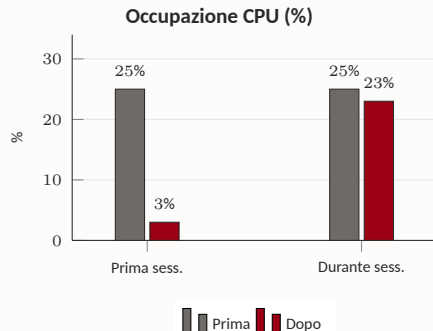
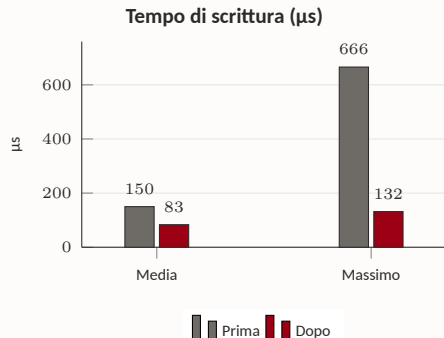


Risultati ottenuti

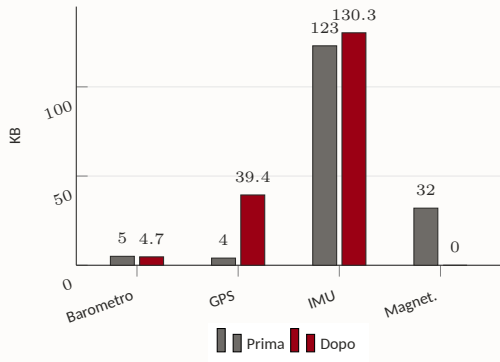
Tempo di scrittura su microSD
Occupazione di CPU e RAM



Risultati: benchmark prestazionali

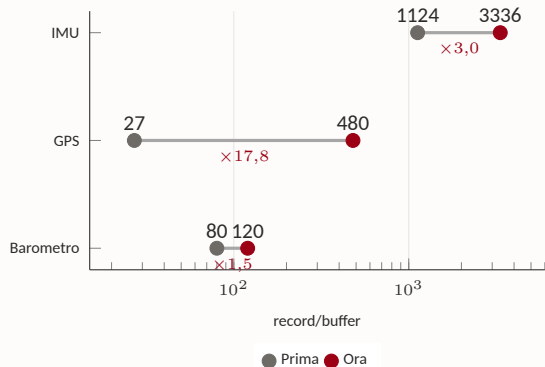


Risultati: occupazione RAM per sensore



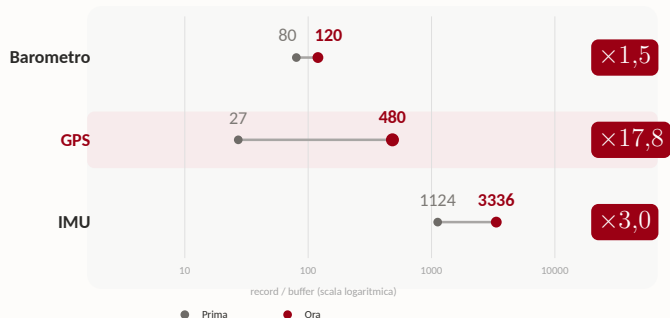
- ▶ Totale buffer: **164,0 KB → 174,4 KB (+6,3%)**
- ▶ GPS e IMU aumentano (+35,4 KB e +7,3 KB): ridurre accessi alla microSD
- ▶ Barometro quasi invariato (−0,3 KB)
- ▶ Magnetometro disabilitato: buffer da 32 KB azzerato

Perché la RAM è aumentata (e il sistema è comunque migliorato)



- ▶ Formato binario: fino a $17,8\times$ più campioni
- ▶ Allocazione posticipata: buffer solo durante la sessione
- ▶ CPU a riposo: dal 25% al 3%
- ▶ Più RAM in cambio di meno scritture su SD

Perché la RAM è aumentata (e il sistema è comunque migliorato)



- ▶ Formato binario: fino a $17,8\times$ più campioni
- ▶ Allocazione posticipata: buffer solo durante la sessione
- ▶ CPU a riposo: dal 25% al 3%
- ▶ Più RAM in cambio di meno scritture su SD

Grazie per l'attenzione

Domande?



Backup: dimensione dei record e dei buffer

Sensore	Prima			Ora		
	Record	Buffer	Rec/buf	Record	Buffer	Rec/buf
Barometro	≈ 32 B	2,5 KB	≈ 80	20 B	2,3 KB	120
GPS	≈ 75 B	2,0 KB	≈ 27	42 B	19,7 KB	480
IMU	≈ 56 B	61,5 KB	≈ 1124	20 B	65,2 KB	3336
Magnetometro	≈ 40 B	16,0 KB	≈ 410	Disabilitato		

Dimensione del record e record/buffer "Prima" approssimati per la variabilità del formato testuale CSV.

Backup: formato dell'header del Superfile

Ogni campo dell'header risponde a una domanda precisa quando il firmware riapre o scrive nel file.

Campo	Funzione
magic / version	Permette di controllare che il file non sia corrotto / il versionamento per future evoluzioni
dataStartOffset	Indicare da quale record iniziano i dati ancora valid
dataEndOffset	Indicare il prossimo record libero
recordCount	Indica quanti record validi sono presenti ora
wrapped	Segnalare se il buffer ha già fatto un giro completo e sta sovrascrivendo dati vecchi